Conditionals, Functions & Loops

AIT 618

Conditionals

Conditionals

- Conditionals allow a programmer to make decisions in the code based on boolean logic
- The most common example of this is through an “if” statement

Conditionals

```
var sum = 100000;
if (process.argv[2] % 2 == 0) {
    sum += process.argv[2];
} else {
    sum -= process.argv[2];
}
```

Conditionals: JavaScript Equals

```
var x = 4
if (x == '4') {
    console.log("It is true!");
} else {
    console.log("It is false!");
}
```

Conditionals: JavaScript Equals (cont.)

- JavaScript has two kinds of equals
  - Double equals (==)
  - Triple equals (===)
- == will only compare VALUE. So ‘4’ == 4 is true
- === will compare both value and data type so ‘4’ === 4 is false

Functions

Functions in JavaScript

- Functions in JavaScript work exactly like methods/functions in other programming languages, but much easier
- There is no need to worry about return types, passing by reference vs. value, etc.

Functions in JavaScript - non void

- The function below takes in two parameters, x and y, adds them together, and returns the sum
- Note: we didn’t need to specify the return type as number

```
function add(x, y) {
    return x + y;
}

add(2,3);
```

Functions in JavaScript - void

- Below is an example of a void function since nothing is returned
- Note that there is no need to specify void

```
function add(x, y) {
    var sum = x + y;
    console.log(sum);
}

add(2,3);
```

Special JavaScript Functions

- Just like with comparison operators, JavaScript has a special type of function

Anonymous Functions

- Anonymous functions are basically the same as normal functions, but there are some advantages to using them depending on the situation:

1. Code brevity, since anonymous functions are not named (a function has no name - Game of Thrones reference)
2. Scope: since they can be used in temporary/private scope
3. Most importantly - are using in callbacks and asynchronous programming since functions are passed as variables
    a. Closures as well, but we will not talk about them in the course

Anonymous Functions Examples

```
function regFunc() {
    console.log("Im a regular function");
}

var anonFunc = function() {
    console.log("Im an anonymous function");
};

regFunc();
anonFunc();
```

Anonymous Function Notes

1. Notice that there is a semi colon after after the function, since we are making an assignment
2. The variable executing the anonymous function is the exact same way is that a normal function is called

Using Anonymous Functions as Variables

```
function regFunc(param1) {
    console.log("Im a regular function");
    param1();
}

var anonFunc = function() {
    console.log("Im an anonymous function");
};

regFunc(anonFunc);
```

Using Anonymous Functions as Variables pt. 2

```
function regFunc(param1) {
    console.log("Im a regular function");
    param1();
}

regFunc(function() {
    console.log("Im an anonymous function");
});
```


COMEDY
CENTRAL

Loops

Loops

- The scholarly definition of a loop is a sequence of instructions that are continually repeated until a certain condition is met.
- Loops exist in all programming languages but vary in how they are defined and used

Loops: Types

- There are many types of loops as well, some that make more sense depending on the job you are trying to accomplish
- Some types of loops:
  - for
  - while
  - do-while
  - for-each
  - for-in
  - etc.
- For this class we will focus on “for” and “forEach”

Loops: Types - for

- For loops are the most common because even though there is “more complexity”, it can tackle almost any task
  - Whether looping through an array of elements
  - Summing the numbers from 1 to n
  - etc.

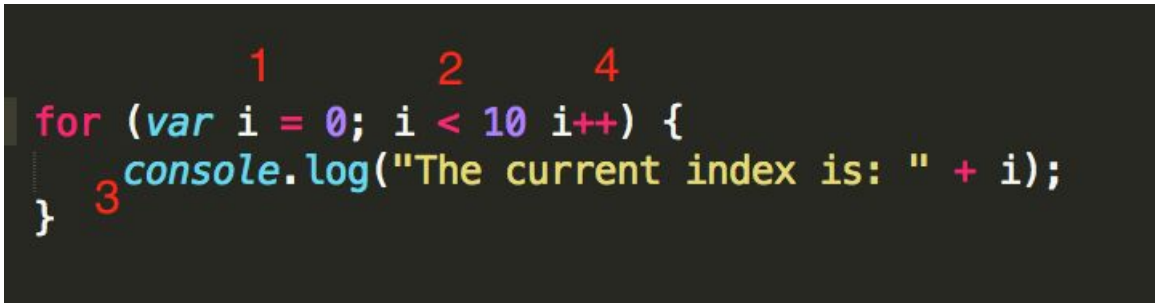
Traditional for-loop Example

- What does the loop below produce?

```
for (var i = 0; i < 10 i++) {
    console.log("The current index is: " + i);
}
```

Traditional for-loop Example

1. Is the initial value the counter is set to
2. Is the condition that is checked every time the loop runs through
3. Is the block of code that will be executed each time the loop runs through
4. Will increment the counter so the loop does not run forever (infinite loop)



Traditional for-loop Using Decrement

- The same way you can increment the counter variable, you can decrement as well
- The loop below will print 10 all the way down to 1

```
for (var i = 10; i > 0; i--) {
    console.log("The current index is: " + i);
}
```

Loop Through a Collection (Array)

- What does the following loop output?

```
var arr = [1,2,3,4,5,6];
for (var i = 0; i < arr.length; i++) {
    console.log(arr[i]);
}
```

Loop Through a Collection (Array cont.)

- What does the following loop output?

```
var arr = [1,2,3,4,5,6];
for (var i = 3; i < arr.length; i++) {
    console.log(arr[i]);
}
```

For loops with a JavaScript Flare

- JavaScript has a special loop called a “forEach” loop that makes it easy to loop through arrays or collections of data
- It does not make you manage a counter or ensure that a certain condition is met
- It simply goes from start to finish of your array and abstracts all of the management details from you

forEach Example

-

```
// Orig
var arr = [1,2,3,4,5,6];
for (var i = 0; i < arr.length; i++) {
    console.log(arr[i]);
}

//New
arr.forEach(function(element) {
    console.log(element);
});
```

forEach Breakdown

```
var arr = [1,2,3,4,5,6];

var anonFunc = function(element) {
    console.log(element);
};

arr.forEach(anonFunc);
```

forEach with Index?

- Notice how the traditional for-loops has an index counter?
- The forEach loop does, but it is hidden until you read the documentation
  - The second parameter is

```
arr.forEach(function(element, i) {
    console.log("Current index: " + i);
});
```